

# CODE SPORT



In this month's column, we continue our discussion on testing machine learning applications.

**L**ast month, we started with an overview of testing machine learning (ML) programs in general. This is an interesting and important field of study, given that ML and AI are starting to have a significant impact on many aspects of human life, including healthcare, transportation, and the legal system. In this month's column, we will delve deeper into the problem of testing ML applications.

Since there were quite a few queries from readers on this topic, I wanted to clarify something first. There has also been considerable interest in using machine learning to improve software testing processes and practices. People have been looking at how test automation can be improved using AI, how ML can help with intelligent testing, and how natural language processing (NLP) can help in automatic test case generation. However, all these topics fall under the broader category of AI driven software testing or AI enabled software testing.

The focus of *our* column is different. We are interested in how the software testing and quality assurance of machine learning applications can be done. Since a few readers had been confused about this topic, I wanted to highlight this distinction first. What we are focusing on is how to test AI and ML application software. We are *not* discussing how AI or ML can help in general software testing, which we will cover in a future article.

In last month's column, we had looked at the machine learning workflow and discussed testing in that context. Now let us consider a different perspective of testing ML applications. What are the various components that need to get tested? For traditional software, testing needs to cover the application code, functional correctness based on the requirements as well as acceptance criteria and

integration testing. For ML applications, the testing needs to encompass not just the code, but also the data. Therefore, we can break down the testing of ML applications into the following stages:

1. Let us first analyse which properties of the ML applications we want to test. These include not just the standard correctness and performance testing, but also the testing of properties such as robustness, fairness, transparency and interpretability.
2. Next, what are the components of the ML applications that we want to test? These include the data, the model code and the framework. For example, let us assume that you are building an application that can do sentiment analysis of the tweets, for which you have decided to use an LSTM based neural network model and the TensorFlow library framework. For this specific case, the data would be tweets, the model would be LSTM based model code that the developer has created, and the framework would be the TensorFlow framework from which the developer is invoking the required functionality. Bugs can reside in each of these components.

Those of you who are familiar with testing traditional software may be surprised to see data being called out as one of the components for testing. Unlike traditional software, data plays a critical role in the inherent correctness of the model. In broader terms, the model consists not just of the model code, but also the parameters/weights of the model which are learnt from the data. From that perspective, feeding of bad data as input to train the model can result in a poor model. Now, what kind of bugs/problems can there be with data?